

zerotrading-core — Execution State Machine

Pass: Execution (Pass 4 of N)

Date: 2026-05-17

Scope: Entry order submission → fill → live position → exit request → exit fill → reconciliation

Constraints: One open position max, hour-harvester strategy only, no UI, no implementation code yet.

1. Overview

The execution layer is a pure state machine. Every incoming event (exchange callback, timer tick, reconciliation result) drives a transition. No side effects happen outside of transitions. All accounting hooks are explicit at each transition, never inferred.

The machine has two **parallel** tracks that run concurrently during a live position:

- **Entry track** — manages the open-order → filled lifecycle
- **Exit track** — manages the exit-order → flat lifecycle (activated once entry is confirmed)

They are not truly independent state machines; they share a single `ExecutionContext` and the exit track cannot start until the entry track reaches `POSITION_OPEN`.

2. TypeScript Types

2.1 State Names

```
/**
 * All states the execution machine can be in.
 * Prefix convention:
 *  IDLE_*      - no active order or position
 *  ENTRY_*    - working an entry order
 *  POSITION_*   - entry confirmed, holding a position
 *  EXIT_*     - working an exit order
 *  TERMINAL_* - cycle complete, no further transitions
 */
export type ExecutionState =

  // — Idle —————
  | 'IDLE'                                // No position, no order. Start state.

  // — Entry track —————
  | 'ENTRY_SUBMITTING'                   // POST /orders in-flight
  | 'ENTRY_OPEN'                         // Order acknowledged, waiting for fills
  | 'ENTRY_PARTIAL'                      // ≥1 fill, quantity not complete
  | 'ENTRY_CANCEL_PENDING'               // DELETE /orders in-flight (timeout or abort)
  | 'ENTRY_RECONCILING'                 // Order cancelled; fetching final fill set

  // — Live position —————
  | 'POSITION_OPEN'                     // Fully (or acceptably) filled; monitoring exit conditions

  // — Exit track —————
  | 'EXIT_EVALUATING'                   // Strategy has requested exit; computing exit price
  | 'EXIT_SUBMITTING'                   // POST /orders in-flight for exit order
  | 'EXIT_OPEN'                         // Exit order acknowledged, waiting for fills
  | 'EXIT_PARTIAL'                      // ≥1 exit fill, still have residual position
  | 'EXIT_CANCEL_PENDING'               // DELETE /orders in-flight
  | 'EXIT_REPRICING'                   // Old exit cancelled; computing new limit price
  | 'EXIT_RECONCILING'                 // Exit order closed; fetching final fill set

  // — Terminal —————
  | 'TERMINAL_FLAT'                     // Position fully exited and accounting settled
  | 'TERMINAL_ERROR';                  // Unrecoverable error; human review required
```

2.2 Exit Modes

```
export type ExitMode =  
  | 'normal'    // Work a limit order; tolerate partial fills; allow brief repricing  
  | 'urgent';   // Cancel and reprice aggressively until flat; no patience budget
```

Normal exit: Place limit at current best-offer side. If unfilled after `normalExitTimeoutMs` (e.g. 20s), cancel and reprice once. If still partially unfilled after second timeout, escalate to urgent.

Urgent exit: Place limit at cross price. If unfilled after `urgentRepricePollMs` (e.g. 3s), cancel and immediately reprice. Repeat up to `urgentMaxReprices` times. After exhausting reprices, emit `REPRICE_LIMIT_EXCEEDED` and go to `TERMINAL_ERROR`.

2.3 Events

```
/**  
 * All events that can drive a transition.  
 * Inputs come from: exchange API responses, polling callbacks, strategy signals, reconciliation.  
 */  
export type ExecutionEvent =  
  // — Strategy signals —————  
  | { type: 'STRATEGY_ENTRY_SIGNAL'; marketTicker: string; side: 'yes' | 'no'; targetCount: number; li  
  | { type: 'STRATEGY_EXIT_SIGNAL'; mode: ExitMode }  
  | { type: 'STRATEGY_ESCALATE_URGENT' }           // Strategy upgrades exit to urgent mid-work  
  
  // — Exchange API responses —————  
  | { type: 'ORDER_SUBMIT_OK'; orderId: string; exchangeStatus: string }  
  | { type: 'ORDER_SUBMIT_FAIL'; error: string }  
  | { type: 'ORDER_CANCEL_OK'; orderId: string }  
  | { type: 'ORDER_CANCEL_FAIL'; orderId: string; error: string }  
  
  // — Fill polling —————  
  | { type: 'FILLS_RECEIVED'; orderId: string; fills: RawFill[] }  
  | { type: 'FILLS_POLL_ERROR'; orderId: string; error: string }  
  
  // — Timers —————  
  | { type: 'ENTRY_TIMEOUT' }                     // Entry order sat unfilled too long  
  | { type: 'EXIT_TIMEOUT' }                       // Normal exit order sat unfilled too long  
  | { type: 'URGENT_REPRICE_TICK' }                // Urgent mode reprice interval fired  
  
  // — Reconciliation —————  
  | { type: 'RECONCILE_FILLS_COMPLETE'; orderId: string; fills: RawFill[]; discrepancy: boolean }  
  | { type: 'RECONCILE_POSITION_COMPLETE'; positionData: ExchangePositionSnapshot }  
  
  // — Market / session —————  
  | { type: 'MARKET_EXPIRY_IMMINENT' }             // Market closes in < N minutes; force exit  
  | { type: 'ENGINE_SHUTDOWN' }                     // Graceful shutdown requested  
  | { type: 'ENGINE_RESTORE'; snapshot: ExecutionSnapshot } // Hot-restart from persisted state  
  
  // — Error / circuit breaker —————  
  | { type: 'CIRCUIT_BREAKER_TRIP' }  
  | { type: 'UNRECOVERABLE_ERROR'; reason: string };
```

2.4 Core Context Types

```
/**  
 * Raw fill as returned by the Kalshi fills API.  
 * Lives in the adapter layer. Never enriched here.  
 */  
export interface RawFill {  
  fill_id: string;  
  order_id: string;  
  market_ticker: string;  
  side: 'yes' | 'no';  
  count_fp: string;           // fixed-point integer string (+10000 = contracts)
```

```

    price: string;           // dollars, e.g. "0.63"
    fee_cost: string;        // dollars, e.g. "0.0027"
    is_taker: boolean;
    created_time: string;    // ISO8601
}

/**
 * Snapshot of a Kalshi MarketPosition returned by GET /portfolio/positions.
 * Used during reconciliation and settlement.
 */
export interface ExchangePositionSnapshot {
    market_ticker: string;
    position_fp: string;      // net contracts in fixed-point
    realized_pnl_dollars: string | null;
    fees_paid_dollars: string | null;
    settled: boolean;
}

/**
 * The order currently being worked by the execution machine.
 * Reset on each new order submission; replaced on cancel/reprice.
 */
export interface ActiveOrderContext {
    orderId: string;
    side: 'yes' | 'no';
    marketTicker: string;
    requestedCount: number;    // contracts requested
    filledCount: number;       // contracts confirmed filled so far
    limitPriceDollars: number;
    submittedAt: number;       // unix ms
    fills: RawFill[];          // raw fills received so far for this order
    cancelAttempts: number;    // how many cancel calls have been sent
    repriceCount: number;      // how many times this order has been repriced
}

/**
 * The full execution context – everything the reducer needs.
 * This is what gets persisted to disk on every transition.
 */
export interface ExecutionContext {
    // Identity
    cycleId: string;           // unique per entry→exit cycle
    marketTicker: string;
    side: 'yes' | 'no';

    // State machine
    state: ExecutionState;
    exitMode: ExitMode | null;

    // Current order being worked (null when no order is open)
    activeOrder: ActiveOrderContext | null;

    // Position tracking (filled once ENTRY_PARTIAL first fill arrives)
    entryFills: RawFill[];     // all confirmed entry fills
    exitFills: RawFill[];      // all confirmed exit fills
    netContractsHeld: number;   // live count; entry fills - exit fills
    entryLimitPriceDollars: number | null;

    // Reprice state
    urgentRepriceCount: number;
    urgentMaxReprices: number; // from config

    // Timestamps
    cycleStartedAt: number;
    positionOpenedAt: number | null;
    exitRequestedAt: number | null;
    cycleClosedAt: number | null;

    // Terminal state metadata
    terminalReason: string | null;
}

```

```
/**
 * Snapshot for hot-restart (ENGINE_RESTORE).
 * Same shape as ExecutionContext; typed alias to make intent explicit.
 */
export type ExecutionSnapshot = ExecutionContext;
```

2.5 Effects (Side Effects Emitted by Transitions)

```
/**
 * Effects are returned by the reducer alongside the next state.
 * The engine runtime processes them after the state is persisted.
 * Never execute effects inside the reducer.
 */
export type ExecutionEffect =
  // Exchange calls
  | { effect: 'SUBMIT_ORDER'; side: 'yes' | 'no'; count: number; limitPriceDollars: number; marketTic
  | { effect: 'CANCEL_ORDER'; orderId: string }
  | { effect: 'POLL_FILLS'; orderId: string; delayMs: number }
  | { effect: 'POLL_POSITION'; marketTicker: string; delayMs: number }

  // Timers
  | { effect: 'SET_ENTRY_TIMEOUT'; delayMs: number }
  | { effect: 'CLEAR_ENTRY_TIMEOUT' }
  | { effect: 'SET_EXIT_TIMEOUT'; delayMs: number; mode: ExitMode }
  | { effect: 'CLEAR_EXIT_TIMEOUT' }
  | { effect: 'SET_URGENT_REPRICE_TICK'; delayMs: number }
  | { effect: 'CLEAR_URGENT_REPRICE_TICK' }

  // Accounting hooks (these are instructions to the accounting layer, not direct calls)
  | { effect: 'ACCOUNTING_RECORD_ENTRY_FILL'; fill: RawFill }
  | { effect: 'ACCOUNTING_RECORD_EXIT_FILL'; fill: RawFill }
  | { effect: 'ACCOUNTING_MARK_PENDING_EXIT'; estimatedExitPriceDollars: number; contractsToExit: num
  | { effect: 'ACCOUNTING_CLEAR_PENDING_EXIT' }
  | { effect: 'ACCOUNTING_RECONCILE_FILLS'; orderId: string; fills: RawFill[]; isExit: boolean }
  | { effect: 'ACCOUNTING_RECONCILE_POSITION'; snapshot: ExchangePositionSnapshot }
  | { effect: 'ACCOUNTING_BOOK_CYCLE_CLOSED'; cycleId: string }

  // Events / audit
  | { effect: 'EMIT_EVENT'; eventType: string; payload: Record<string, unknown> }
  | { effect: 'PERSIST_CONTEXT'; context: ExecutionContext }
  | { effect: 'ALERT_HUMAN'; reason: string; context: ExecutionContext };
```

3. Transition Table

Format: Current State + Event → Next State | Key Effects

Entry Track

Current State	Event	Next State	Key Effects
IDLE	STRATEGY_ENTRY_SIGNAL	ENTRY_SUBMITTING	SUBMIT_ORDER, PERSIST_CONTEXT, EMIT_EVENT(entry_signal)
ENTRY_SUBMITTING	ORDER_SUBMIT_OK	ENTRY_OPEN	SET_ENTRY_TIMEOUT, POLL_FILLS (2000ms), PERSIST_CONTEXT
ENTRY_SUBMITTING	ORDER_SUBMIT_FAIL	TERMINAL_ERROR	EMIT_EVENT(entry_submit_fail), ALERT_HUMAN, PERSIST_CONTEXT
ENTRY_OPEN	FILLS_RECEIVED(partial)	ENTRY_PARTIAL	ACCOUNTING_RECORD_ENTRY_FILL per fill, POLL_FILLS (2000ms), PERSIST_CONTEXT
ENTRY_OPEN	FILLS_RECEIVED(complete)	POSITION_OPEN	ACCOUNTING_RECORD_ENTRY_FILL per fill, CLEAR_ENTRY_TIMEOUT, EMIT_EVENT(position_opened), PERSIST_CONTEXT

Current State	Event	Next State	Key Effects
ENTRY_OPEN	ENTRY_TIMEOUT	ENTRY_CANCEL_PENDING	CANCEL_ORDER, EMIT_EVENT(entry_timeout), PERSIST_CONTEXT
ENTRY_OPEN	CIRCUIT_BREAKER_TRIP	ENTRY_CANCEL_PENDING	CANCEL_ORDER, CLEAR_ENTRY_TIMEOUT, EMIT_EVENT(circuit_breaker_cancel_entry), PERSIST_CONTEXT
ENTRY_OPEN	ENGINE_SHUTDOWN	ENTRY_CANCEL_PENDING	CANCEL_ORDER, CLEAR_ENTRY_TIMEOUT, PERSIST_CONTEXT
ENTRY_PARTIAL	FILLS_RECEIVED(more fills)	ENTRY_PARTIAL (or POSITION_OPEN if complete)	ACCOUNTING_RECORD_ENTRY_FILL, POLL_FILLS (2000ms), PERSIST_CONTEXT
ENTRY_PARTIAL	ENTRY_TIMEOUT	ENTRY_CANCEL_PENDING	CANCEL_ORDER, EMIT_EVENT(entry_partial_timeout), PERSIST_CONTEXT
ENTRY_CANCEL_PENDING	ORDER_CANCEL_OK	ENTRY_RECONCILING	POLL_FILLS (500ms), EMIT_EVENT(entry_cancel_ack), PERSIST_CONTEXT
ENTRY_CANCEL_PENDING	ORDER_CANCEL_FAIL	ENTRY_CANCEL_PENDING	CANCEL_ORDER (retry if attempts < 3), else → TERMINAL_ERROR with ALERT_HUMAN
ENTRY_CANCEL_PENDING	FILLS_RECEIVED(fills arrive during cancel)	ENTRY_CANCEL_PENDING	ACCOUNTING_RECORD_ENTRY_FILL, accumulate; do not change state — cancel still in-flight
ENTRY_RECONCILING	RECONCILE_FILLS_COMPLETE (0 fills)	TERMINAL_FLAT	ACCOUNTING_BOOK_CYCLE_CLOSED, EMIT_EVENT(cycle_abandoned_no_fill), PERSIST_CONTEXT
ENTRY_RECONCILING	RECONCILE_FILLS_COMPLETE (≥1 fill)	POSITION_OPEN	ACCOUNTING_RECONCILE_FILLS, EMIT_EVENT(position_opened_partial_entry), PERSIST_CONTEXT

Live Position → Exit Track

Current State	Event	Next State	Key Effects
POSITION_OPEN	STRATEGY_EXIT_SIGNAL	EXIT_EVALUATING	EMIT_EVENT(exit_requested), PERSIST_CONTEXT
POSITION_OPEN	MARKET_EXPIRY_IMMINENT	EXIT_EVALUATING (mode forced urgent)	EMIT_EVENT(market_expiry_force_exit), PERSIST_CONTEXT
POSITION_OPEN	ENGINE_SHUTDOWN	EXIT_EVALUATING (mode urgent)	EMIT_EVENT(shutdown_force_exit), PERSIST_CONTEXT
POSITION_OPEN	CIRCUIT_BREAKER_TRIP	EXIT_EVALUATING (mode urgent)	EMIT_EVENT(circuit_breaker_force_exit), PERSIST_CONTEXT
EXIT_EVALUATING	<i>(internal: price computed)</i>	EXIT_SUBMITTING	ACCOUNTING_MARK_PENDING_EXIT, SUBMIT_ORDER, PERSIST_CONTEXT
EXIT_SUBMITTING	ORDER_SUBMIT_OK	EXIT_OPEN	SET_EXIT_TIMEOUT(mode), POLL_FILLS (2000ms), PERSIST_CONTEXT
EXIT_SUBMITTING	ORDER_SUBMIT_FAIL	EXIT_EVALUATING (retry once)	EMIT_EVENT(exit_submit_fail), PERSIST_CONTEXT; if 2nd fail → TERMINAL_ERROR, ALERT_HUMAN
EXIT_OPEN	FILLS_RECEIVED(partial)	EXIT_PARTIAL	ACCOUNTING_RECORD_EXIT_FILL, POLL_FILLS (2000ms), PERSIST_CONTEXT
EXIT_OPEN	FILLS_RECEIVED(complete)	EXIT_RECONCILING	ACCOUNTING_RECORD_EXIT_FILL, CLEAR_EXIT_TIMEOUT, POLL_FILLS (500ms), PERSIST_CONTEXT

Current State	Event	Next State	Key Effects
EXIT_OPEN	EXIT_TIMEOUT (normal mode)	EXIT_CANCEL_PENDING	CANCEL_ORDER, EMIT_EVENT(exit_normal_timeout), PERSIST_CONTEXT
EXIT_OPEN	URGENT_REPRICE_TICK (urgent mode)	EXIT_CANCEL_PENDING	CANCEL_ORDER, EMIT_EVENT(exit_urgent_reprice), PERSIST_CONTEXT
EXIT_OPEN	STRATEGY_ESCALATE_URGENT	EXIT_CANCEL_PENDING	CANCEL_ORDER, setexitMode = urgent, EMIT_EVENT(exit_escalated_urgent), PERSIST_CONTEXT
EXIT_PARTIAL	FILLS_RECEIVED (more fills)	EXIT_PARTIAL (or EXIT_RECONCILING if flat)	ACCOUNTING_RECORD_EXIT_FILL, POLL_FILLS (2000ms), PERSIST_CONTEXT
EXIT_PARTIAL	EXIT_TIMEOUT (normal mode)	EXIT_CANCEL_PENDING	CANCEL_ORDER, EMIT_EVENT(exit_partial_timeout), PERSIST_CONTEXT
EXIT_PARTIAL	URGENT_REPRICE_TICK (urgent mode)	EXIT_CANCEL_PENDING	CANCEL_ORDER, EMIT_EVENT(exit_urgent_partial_reprice), PERSIST_CONTEXT
EXIT_CANCEL_PENDING	ORDER_CANCEL_OK	EXIT_REPRICING	POLL_FILLS (500ms), EMIT_EVENT(exit_cancel_ack), PERSIST_CONTEXT
EXIT_CANCEL_PENDING	ORDER_CANCEL_FAIL	EXIT_CANCEL_PENDING	Retry if attempts < 3; else → TERMINAL_ERROR, ALERT_HUMAN
EXIT_CANCEL_PENDING	FILLS_RECEIVED (fills arrive during cancel)	EXIT_CANCEL_PENDING	ACCOUNTING_RECORD_EXIT_FILL; if now flat, set flag to transition → EXIT_RECONCILING after cancel ACK
EXIT_REPRICING	<i>(internal: all contracts filled from cancel ACK fills)</i>	EXIT_RECONCILING	CLEAR_EXIT_TIMEOUT, ACCOUNTING_CLEAR_PENDING_EXIT, PERSIST_CONTEXT
EXIT_REPRICING	<i>(internal: residual contracts remain)</i>	EXIT_SUBMITTING	Compute new price, SUBMIT_ORDER, guard urgentRepriceCount, PERSIST_CONTEXT
EXIT_REPRICING	urgentRepriceCount >= urgentMaxReprices (guard)	TERMINAL_ERROR	ALERT_HUMAN (reprice_limit_exceeded), ACCOUNTING_CLEAR_PENDING_EXIT, PERSIST_CONTEXT
EXIT_RECONCILING	RECONCILE_FILLS_COMPLETE	TERMINAL_FLAT	ACCOUNTING_RECONCILE_FILLS, ACCOUNTING_RECONCILE_POSITION, ACCOUNTING_BOOK_CYCLE_CLOSED, EMIT_EVENT(cycle_closed), PERSIST_CONTEXT

Engine-Level Transitions (Any State)

From	Event	Next State	Key Effects
Any	UNRECOVERABLE_ERROR	TERMINAL_ERROR	ALERT_HUMAN, PERSIST_CONTEXT
Any	ENGINE_RESTORE	Restored from snapshot	Validate snapshot; resume timers and polls; EMIT_EVENT(engine_restored)

4. Reducer Pseudocode

```
/**
 * Pure transition function. No I/O. No side effects. Returns next state + effects.
 * The caller (engine runtime) persists context and executes effects.
 */
function transition(
  ctx: ExecutionContext,
  event: ExecutionEvent
```

```

): [ExecutionContext, ExecutionEffect[]] {

  const effects: ExecutionEffect[] = [];

  // — Helper to clone context —————
  function next(patch: Partial<ExecutionContext>): ExecutionContext {
    return { ...ctx, ...patch };
  }

  // — IDLE —————
  if (ctx.state === 'IDLE' && event.type === 'STRATEGY_ENTRY_SIGNAL') {
    const orderId = generateCycleId(); // will be filled by ORDER_SUBMIT_OK
    const activeOrder: ActiveOrderContext = {
      orderId: '', // unknown until submit response
      side: event.side,
      marketTicker: event.marketTicker,
      requestedCount: event.targetCount,
      filledCount: 0,
      limitPriceDollars: event.limitPriceDollars,
      submittedAt: Date.now(),
      fills: [],
      cancelAttempts: 0,
      repriceCount: 0,
    };
    effects.push({ effect: 'SUBMIT_ORDER', side: event.side, count: event.targetCount, limitPriceDollars: event.limitPriceDollars });
    effects.push({ effect: 'EMIT_EVENT', eventType: 'entry_signal', payload: { ...event } });
    const nextCtx = next({ state: 'ENTRY_SUBMITTING', activeOrder, cycleStartedAt: Date.now(), marketTicker: event.marketTicker });
    effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
    return [nextCtx, effects];
  }

  // — ENTRY_SUBMITTING —————
  if (ctx.state === 'ENTRY_SUBMITTING' && event.type === 'ORDER_SUBMIT_OK') {
    const activeOrder = { ...ctx.activeOrder!, orderId: event.orderId };
    effects.push({ effect: 'SET_ENTRY_TIMEOUT', delayMs: config.entryTimeoutMs });
    effects.push({ effect: 'POLL_FILLS', orderId: event.orderId, delayMs: 2000 });
    const nextCtx = next({ state: 'ENTRY_OPEN', activeOrder });
    effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
    return [nextCtx, effects];
  }

  if (ctx.state === 'ENTRY_SUBMITTING' && event.type === 'ORDER_SUBMIT_FAIL') {
    const nextCtx = next({ state: 'TERMINAL_ERROR', terminalReason: `entry_submit_fail: ${event.error}` });
    effects.push({ effect: 'EMIT_EVENT', eventType: 'entry_submit_fail', payload: { error: event.error } });
    effects.push({ effect: 'ALERT_HUMAN', reason: 'Entry order submission failed', context: nextCtx });
    effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
    return [nextCtx, effects];
  }

  // — ENTRY_OPEN / ENTRY_PARTIAL — fill receipts —————
  if ((ctx.state === 'ENTRY_OPEN' || ctx.state === 'ENTRY_PARTIAL') && event.type === 'FILLS_RECEIVED') {
    // Deduplicate: only process fills not already in entryFills
    const knownIds = new Set(ctx.entryFills.map(f => f.fill_id));
    const newFills = event.fills.filter(f => !knownIds.has(f.fill_id));

    let updatedEntryFills = [...ctx.entryFills, ...newFills];
    let filledCount = updatedEntryFills.reduce((sum, f) => sum + parseFP(f.count_fp), 0);
    const activeOrder = { ...ctx.activeOrder!, filledCount, fills: [...ctx.activeOrder!.fills, ...newFills] };

    for (const fill of newFills) {
      effects.push({ effect: 'ACCOUNTING_RECORD_ENTRY_FILL', fill });
    }

    const isComplete = filledCount >= ctx.activeOrder!.requestedCount;
    if (isComplete) {
      effects.push({ effect: 'CLEAR_ENTRY_TIMEOUT' });
      effects.push({ effect: 'EMIT_EVENT', eventType: 'position_opened', payload: { filledCount } });
      const nextCtx = next({ state: 'POSITION_OPEN', entryFills: updatedEntryFills, netContractsHeld: ctx.netContractsHeld });
      effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
      return [nextCtx, effects];
    } else {
      effects.push({ effect: 'POLL_FILLS', orderId: ctx.activeOrder!.orderId, delayMs: 2000 });
    }
  }
}

```

```

    const nextCtx = next({ state: 'ENTRY_PARTIAL', entryFills: updatedEntryFills, netContractsHeld: fi
    effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
    return [nextCtx, effects];
  }
}

// — ENTRY_OPEN — timeout —————
if (ctx.state === 'ENTRY_OPEN' && event.type === 'ENTRY_TIMEOUT') {
  effects.push({ effect: 'CANCEL_ORDER', orderId: ctx.activeOrder!.orderId });
  effects.push({ effect: 'EMIT_EVENT', eventType: 'entry_timeout', payload: { orderId: ctx.activeOrd
  const activeOrder = { ...ctx.activeOrder!, cancelAttempts: ctx.activeOrder!.cancelAttempts + 1 };
  const nextCtx = next({ state: 'ENTRY_CANCEL_PENDING', activeOrder });
  effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
  return [nextCtx, effects];
}

// — ENTRY_PARTIAL — timeout —————
if (ctx.state === 'ENTRY_PARTIAL' && event.type === 'ENTRY_TIMEOUT') {
  // We have some fills – cancel remainder and proceed with what we have
  effects.push({ effect: 'CANCEL_ORDER', orderId: ctx.activeOrder!.orderId });
  effects.push({ effect: 'EMIT_EVENT', eventType: 'entry_partial_timeout', payload: { filledCount: ct
  const activeOrder = { ...ctx.activeOrder!, cancelAttempts: ctx.activeOrder!.cancelAttempts + 1 };
  const nextCtx = next({ state: 'ENTRY_CANCEL_PENDING', activeOrder });
  effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
  return [nextCtx, effects];
}

// — ENTRY_CANCEL_PENDING —————
if (ctx.state === 'ENTRY_CANCEL_PENDING' && event.type === 'ORDER_CANCEL_OK') {
  effects.push({ effect: 'POLL_FILLS', orderId: ctx.activeOrder!.orderId, delayMs: 500 });
  effects.push({ effect: 'EMIT_EVENT', eventType: 'entry_cancel_ack', payload: { orderId: ctx.activeO
  const nextCtx = next({ state: 'ENTRY_RECONCILING' });
  effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
  return [nextCtx, effects];
}

if (ctx.state === 'ENTRY_CANCEL_PENDING' && event.type === 'ORDER_CANCEL_FAIL') {
  const attempts = ctx.activeOrder!.cancelAttempts;
  if (attempts < 3) {
    effects.push({ effect: 'CANCEL_ORDER', orderId: ctx.activeOrder!.orderId });
    const activeOrder = { ...ctx.activeOrder!, cancelAttempts: attempts + 1 };
    const nextCtx = next({ activeOrder });
    effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
    return [nextCtx, effects];
  } else {
    const nextCtx = next({ state: 'TERMINAL_ERROR', terminalReason: 'entry_cancel_failed_3_attempts' });
    effects.push({ effect: 'ALERT_HUMAN', reason: 'Entry cancel failed after 3 attempts', context: ne
    effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
    return [nextCtx, effects];
  }
}

// — ENTRY_RECONCILING —————
if (ctx.state === 'ENTRY_RECONCILING' && event.type === 'RECONCILE_FILLS_COMPLETE') {
  effects.push({ effect: 'ACCOUNTING_RECONCILE_FILLS', orderId: ctx.activeOrder!.orderId, fills: ev
  if (event.fills.length === 0) {
    // Cancelled with no fills – clean abort
    const nextCtx = next({ state: 'TERMINAL_FLAT', activeOrder: null, cycleClosedAt: Date.now() });
    effects.push({ effect: 'ACCOUNTING_BOOK_CYCLE_CLOSED', cycleId: ctx.cycleId });
    effects.push({ effect: 'EMIT_EVENT', eventType: 'cycle_abandoned_no_fill', payload: {} });
    effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
    return [nextCtx, effects];
  } else {
    const filledCount = event.fills.reduce((sum, f) => sum + parseFP(f.count_fp), 0);
    effects.push({ effect: 'EMIT_EVENT', eventType: 'position_opened_partial_entry', payload: { fille
    const nextCtx = next({ state: 'POSITION_OPEN', entryFills: event.fills, netContractsHeld: filledC
    effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
    return [nextCtx, effects];
  }
}

// — POSITION_OPEN — exit triggers —————

```

```

if (ctx.state === 'POSITION_OPEN' && (
  event.type === 'STRATEGY_EXIT_SIGNAL' ||
  event.type === 'MARKET_EXPIRY_IMMINENT' ||
  event.type === 'ENGINE_SHUTDOWN' ||
  event.type === 'CIRCUIT_BREAKER_TRIP'
)) {
  const mode: ExitMode =
    event.type === 'STRATEGY_EXIT_SIGNAL' ? event.mode :
    'urgent'; // forced exits are always urgent
  effects.push({ effect: 'EMIT_EVENT', eventType: 'exit_requested', payload: { mode, trigger: event });
  const nextCtx = next({ state: 'EXIT_EVALUATING', exitMode: mode, exitRequestedAt: Date.now() });
  effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
  return [nextCtx, effects];
}

// — EXIT_EVALUATING — price computed (internal trigger, driven by engine) —
// NOTE: This state is exited by the engine calling transition() with a
// synthetic internal event after computing the exit price. In practice,
// EXIT_EVALUATING is a transient state; the engine resolves it synchronously.
// We model it as a separate state so that a crash during price computation
// is recoverable (we know we need to recompute and re-submit).
if (ctx.state === 'EXIT_EVALUATING' && event.type === 'ORDER_SUBMIT_OK') {
  // Engine computed exit price, submitted, and now we get the ACK
  // (The SUBMIT_ORDER effect was emitted by the engine's EXIT_EVALUATING handler)
  const activeOrder: ActiveOrderContext = {
    orderId: event.orderId,
    side: ctx.side === 'yes' ? 'no' : 'yes', // exit is opposite side
    marketTicker: ctx.marketTicker,
    requestedCount: ctx.netContractsHeld,
    filledCount: 0,
    limitPriceDollars: 0, // engine fills this before emitting the event
    submittedAt: Date.now(),
    fills: [],
    cancelAttempts: 0,
    repriceCount: 0,
  };
  const timeoutMs = ctx.exitMode === 'urgent' ? config.urgentRepricePollMs : config.normalExitTimeoutMs;
  effects.push(ctx.exitMode === 'urgent'
    ? { effect: 'SET_URGENT_REPRICE_TICK', delayMs: timeoutMs }
    : { effect: 'SET_EXIT_TIMEOUT', delayMs: timeoutMs, mode: 'normal' }
  );
  effects.push({ effect: 'POLL_FILLS', orderId: event.orderId, delayMs: 2000 });
  const nextCtx = next({ state: 'EXIT_OPEN', activeOrder });
  effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
  return [nextCtx, effects];
}

// — EXIT_OPEN / EXIT_PARTIAL — fill receipts —————
if ((ctx.state === 'EXIT_OPEN' || ctx.state === 'EXIT_PARTIAL') && event.type === 'FILLS_RECEIVED') {
  const knownIds = new Set(ctx.exitFills.map(f => f.fill_id));
  const newFills = event.fills.filter(f => !knownIds.has(f.fill_id));

  const updatedExitFills = [...ctx.exitFills, ...newFills];
  const exitFilledCount = updatedExitFills.reduce((sum, f) => sum + parseFP(f.count_fp), 0);
  const residual = ctx.netContractsHeld - exitFilledCount;
  const activeOrder = { ...ctx.activeOrder!, filledCount: exitFilledCount, fills: [...ctx.activeOrder!.fills, ...newFills] };

  for (const fill of newFills) {
    effects.push({ effect: 'ACCOUNTING_RECORD_EXIT_FILL', fill });
  }

  if (residual <= 0) {
    // Fully exited
    effects.push({ effect: 'CLEAR_EXIT_TIMEOUT' });
    effects.push({ effect: 'CLEAR_URGENT_REPRICE_TICK' });
    effects.push({ effect: 'ACCOUNTING_CLEAR_PENDING_EXIT' });
    effects.push({ effect: 'POLL_FILLS', orderId: ctx.activeOrder!.orderId, delayMs: 500 });
    const nextCtx = next({ state: 'EXIT_RECONCILING', exitFills: updatedExitFills, netContractsHeld: residual });
    effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
    return [nextCtx, effects];
  } else {
    effects.push({ effect: 'POLL_FILLS', orderId: ctx.activeOrder!.orderId, delayMs: 2000 });
  }
}

```

```

    const nextCtx = next({ state: 'EXIT_PARTIAL', exitFills: updatedExitFills, netContractsHeld: resic
    effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
    return [nextCtx, effects];
  }
}

// — EXIT_OPEN — timeout / reprice tick —————
if (ctx.state === 'EXIT_OPEN' && (event.type === 'EXIT_TIMEOUT' || event.type === 'URGENT_REPRICE_TICK
effects.push({ effect: 'CANCEL_ORDER', orderId: ctx.activeOrder!.orderId });
effects.push({ effect: 'EMIT_EVENT', eventType: 'exit_cancel_for_reprice', payload: { mode: ctx.ex
const activeOrder = { ...ctx.activeOrder!, cancelAttempts: 1 };
const nextCtx = next({ state: 'EXIT_CANCEL_PENDING', activeOrder });
effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
return [nextCtx, effects];
}

// — EXIT_OPEN — escalation —————
if (ctx.state === 'EXIT_OPEN' && event.type === 'STRATEGY_ESCALATE_URGENT') {
  effects.push({ effect: 'CANCEL_ORDER', orderId: ctx.activeOrder!.orderId });
  effects.push({ effect: 'CLEAR_EXIT_TIMEOUT' }); // clear normal timeout
  effects.push({ effect: 'EMIT_EVENT', eventType: 'exit_escalated_urgent', payload: {} });
  const activeOrder = { ...ctx.activeOrder!, cancelAttempts: 1 };
  const nextCtx = next({ state: 'EXIT_CANCEL_PENDING', activeOrder, exitMode: 'urgent' });
  effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
  return [nextCtx, effects];
}

// — EXIT_CANCEL_PENDING —————
if (ctx.state === 'EXIT_CANCEL_PENDING' && event.type === 'ORDER_CANCEL_OK') {
  effects.push({ effect: 'POLL_FILLS', orderId: ctx.activeOrder!.orderId, delayMs: 500 });
  const nextCtx = next({ state: 'EXIT_REPRICING' });
  effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
  return [nextCtx, effects];
}

if (ctx.state === 'EXIT_CANCEL_PENDING' && event.type === 'ORDER_CANCEL_FAIL') {
  const attempts = ctx.activeOrder!.cancelAttempts;
  if (attempts < 3) {
    effects.push({ effect: 'CANCEL_ORDER', orderId: ctx.activeOrder!.orderId });
    const activeOrder = { ...ctx.activeOrder!, cancelAttempts: attempts + 1 };
    const nextCtx = next({ activeOrder });
    effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
    return [nextCtx, effects];
  } else {
    const nextCtx = next({ state: 'TERMINAL_ERROR', terminalReason: 'exit_cancel_failed_3_attempts' });
    effects.push({ effect: 'ALERT_HUMAN', reason: 'Exit cancel failed after 3 attempts – position may
    effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
    return [nextCtx, effects];
  }
}

// Fills arriving during cancel – accumulate, don't change state
if (ctx.state === 'EXIT_CANCEL_PENDING' && event.type === 'FILLS_RECEIVED') {
  const knownIds = new Set(ctx.exitFills.map(f => f.fill_id));
  const newFills = event.fills.filter(f => !knownIds.has(f.fill_id));
  const updatedExitFills = [...ctx.exitFills, ...newFills];
  const exitFilledCount = updatedExitFills.reduce((sum, f) => sum + parseFP(f.count_fp), 0);
  for (const fill of newFills) {
    effects.push({ effect: 'ACCOUNTING_RECORD_EXIT_FILL', fill });
  }
  // Mark on context whether we're already flat (so EXIT_REPRICING can fast-path)
  const alreadyFlat = exitFilledCount >= ctx.netContractsHeld;
  const nextCtx = next({ exitFills: updatedExitFills, netContractsHeld: Math.max(0, ctx.netContractsH
  // Note: alreadyFlat flag should be on context; simplified here for brevity
  effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
  return [nextCtx, effects];
}

// — EXIT_REPRICING —————
// Engine reads EXIT_REPRICING, checks netContractsHeld, computes new price, and emits ORDER_SUBMIT_
if (ctx.state === 'EXIT_REPRICING') {
  // Guard: reprice limit

```

```

    if (ctx.urgentRepriceCount >= ctx.urgentMaxReprices && ctx.exitMode === 'urgent') {
      const nextCtx = next({ state: 'TERMINAL_ERROR', terminalReason: 'reprice_limit_exceeded' });
      effects.push({ effect: 'ACCOUNTING_CLEAR_PENDING_EXIT' });
      effects.push({ effect: 'ALERT_HUMAN', reason: `Urgent exit reprice limit (${ctx.urgentMaxReprice
      effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
      return [nextCtx, effects];
    }

    if (ctx.netContractsHeld <= 0) {
      // Already flat from fills that arrived during cancel
      effects.push({ effect: 'ACCOUNTING_CLEAR_PENDING_EXIT' });
      effects.push({ effect: 'POLL_FILLS', orderId: ctx.activeOrder?.orderId ?? '', delayMs: 200 });
      const nextCtx = next({ state: 'EXIT_RECONCILING', activeOrder: null });
      effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
      return [nextCtx, effects];
    }

    // Engine will compute new exit price and submit. Transition to EXIT_SUBMITTING,
    // then ORDER_SUBMIT_OK → EXIT_OPEN (reuse the existing EXIT_EVALUATING → EXIT_OPEN path).
    // urgentRepriceCount increment happens here.
    const nextCtx = next({ state: 'EXIT_EVALUATING', urgentRepriceCount: ctx.urgentRepriceCount + 1, ac
    effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
    return [nextCtx, effects];
  }

  // — EXIT_RECONCILING —————
  if (ctx.state === 'EXIT_RECONCILING' && event.type === 'RECONCILE_FILLS_COMPLETE') {
    effects.push({ effect: 'ACCOUNTING_RECONCILE_FILLS', orderId: ctx.activeOrder?.orderId ?? '', fil
    effects.push({ effect: 'POLL_POSITION', marketTicker: ctx.marketTicker, delayMs: 1000 });
    // Position reconciliation fires next; RECONCILE_POSITION_COMPLETE below closes the cycle
    const nextCtx = next({ exitFills: event.fills });
    effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
    return [nextCtx, effects];
  }

  if (ctx.state === 'EXIT_RECONCILING' && event.type === 'RECONCILE_POSITION_COMPLETE') {
    effects.push({ effect: 'ACCOUNTING_RECONCILE_POSITION', snapshot: event.positionData });
    effects.push({ effect: 'ACCOUNTING_BOOK_CYCLE_CLOSED', cycleId: ctx.cycleId });
    effects.push({ effect: 'EMIT_EVENT', eventType: 'cycle_closed', payload: { cycleId: ctx.cycleId } });
    const nextCtx = next({ state: 'TERMINAL_FLAT', cycleClosedAt: Date.now(), activeOrder: null });
    effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
    return [nextCtx, effects];
  }

  // — ENGINE_RESTORE —————
  if (event.type === 'ENGINE_RESTORE') {
    // Validate snapshot is not terminal
    if (event.snapshot.state === 'TERMINAL_FLAT' || event.snapshot.state === 'TERMINAL_ERROR') {
      // Don't restore terminal states; return idle
      const nextCtx: ExecutionContext = { ...event.snapshot, state: 'IDLE' };
      effects.push({ effect: 'EMIT_EVENT', eventType: 'engine_restored_terminal_reset', payload: { pre
      return [nextCtx, effects];
    }
    // Rehydrate: restore timers/polls based on restored state
    rehydrateEffects(event.snapshot, effects); // engine helper, not pure
    effects.push({ effect: 'EMIT_EVENT', eventType: 'engine_restored', payload: { state: event.snapshot
    return [event.snapshot, effects];
  }

  // — Unhandled — should not happen —————
  // In production, this is a TERMINAL_ERROR; during dev, throw.
  const nextCtx = next({ state: 'TERMINAL_ERROR', terminalReason: `unhandled_event: ${event.type} in sta
  effects.push({ effect: 'ALERT_HUMAN', reason: `Unhandled event ${event.type} in state ${ctx.state}`,
  effects.push({ effect: 'PERSIST_CONTEXT', context: nextCtx });
  return [nextCtx, effects];
}

```

5. Persistence Requirements at Each Transition

The principle: **persist before acting**. The engine writes the new `ExecutionContext` to disk before executing any effects. On restart, the engine reads the last-written context and rehydrates.

Transition	What to Persist
IDLE → ENTRY_SUBMITTING	New <code>ExecutionContext</code> with <code>state</code> , <code>cycleId</code> , <code>activeOrder</code> , <code>cycleStartedAt</code>
ENTRY_SUBMITTING → ENTRY_OPEN	Updated <code>activeOrder.orderId</code> ; timer registration
Any fill received	Updated <code>entryFills/exitFills</code> , <code>netContractsHeld</code> , <code>activeOrder.filledCount</code>
Any cancel attempt	Updated <code>activeOrder.cancelAttempts</code>
ENTRY_CANCEL_PENDING → ENTRY_RECONCILING	State transition only
ENTRY_RECONCILING → POSITION_OPEN	<code>entryFills</code> , <code>netContractsHeld</code> , cleared <code>activeOrder</code>
POSITION_OPEN → EXIT_EVALUATING	<code>exitMode</code> , <code>exitRequestedAt</code>
EXIT_EVALUATING → EXIT_OPEN	New <code>activeOrder</code> for exit order; <code>urgentRepriceCount</code> if incremented
EXIT_OPEN/PARTIAL → EXIT_CANCEL_PENDING	State + cancel attempt count
EXIT_CANCEL_PENDING → EXIT_REPRICING	State only
EXIT_REPRICING → EXIT_EVALUATING	<code>urgentRepriceCount</code> incremented
EXIT_RECONCILING → TERMINAL_FLAT	Final <code>exitFills</code> , <code>cycleClosedAt</code> ; cleared <code>activeOrder</code>
Any → TERMINAL_ERROR	Full context + <code>terminalReason</code>

Persistence strategy: Append a JSON line to the NDJSON event log (`ILogEventLog`) for every state transition. Also overwrite a single `execution_context.json` with the current `ExecutionContext`. The event log is the audit trail; the context file is the restart checkpoint.

6. Accounting Hooks at Each Transition

This table maps execution transitions to accounting-layer calls. The accounting layer is separate — the execution machine only emits the effects; accounting processes them asynchronously (but before the next engine cycle).

Transition	Accounting Hook	What It Does
Entry fill received	ACCOUNTING_RECORD_ENTRY_FILL	Creates <code>FillRecord</code> + <code>LedgerEntry(debit_collateral, fee_estimated)</code>
ENTRY_CANCEL_PENDING → ENTRY_RECONCILING	<i>(none — wait for final fills)</i>	—
ENTRY_RECONCILING → POSITION_OPEN	ACCOUNTING_RECONCILE_FILLS(<code>isExit=false</code>)	Creates/updates <code>ReconciliationEvent</code> ; corrects any duplicate fill double-counts
POSITION_OPEN → EXIT_EVALUATING	ACCOUNTING_MARK_PENDING_EXIT	Writes <code>pendingExitEstimate</code> on <code>PositionRecord</code> ; display only, never feeds breaker
Exit fill received	ACCOUNTING_RECORD_EXIT_FILL	Creates <code>FillRecord</code> + <code>LedgerEntry(credit_proceeds, fee_estimated)</code>
EXIT_CANCEL_PENDING → EXIT_REPRICING	ACCOUNTING_CLEAR_PENDING_EXIT (on final reprice or flat)	Clears <code>pendingExitEstimate</code>
EXIT_RECONCILING	ACCOUNTING_RECONCILE_FILLS(<code>isExit=true</code>)	Reconciles exit fills; creates <code>ReconciliationEvent</code> for any discrepancy

Transition	Accounting Hook	What It Does
EXIT_RECONCILING	ACCOUNTING_RECONCILE_POSITION	Pulls realized_pnl_dollars from exchange; creates authoritative LedgerEntry(realized_pnl) and fee_confirmed entries; closes PositionRecord
→ TERMINAL_FLAT	ACCOUNTING_BOOK_CYCLE_CLOSED	Freezes PositionRecord; computes final BreakerAccounting delta
→ TERMINAL_ERROR	<i>(no auto-booking — manual reconciliation required)</i>	Emits ALERT_HUMAN only

Critical accounting invariant from the accounting schema: ACCOUNTING_RECORD_EXIT_FILL must NOT book realized P&L. It only records the fill fact and an estimated fee. ACCOUNTING_RECONCILE_POSITION (after EXIT_RECONCILING) is the only place realized P&L is booked to the authoritative ledger.

7. Edge Cases

7.1 Fills arrive after cancel ACK (race condition)

Scenario: Order cancel ACK arrives, machine moves to ENTRY_RECONCILING or EXIT_REPRICING. Then a late fill arrives for the now-cancelled order.

Handling: The FILLS_RECEIVED event in ENTRY_CANCEL_PENDING / EXIT_CANCEL_PENDING accumulates fills and records them to accounting. In ENTRY_RECONCILING, RECONCILE_FILLS_COMPLETE does a final authoritative poll that catches any late fills. Because fill deduplication uses fill_id, double-processing is prevented.

The RECONCILE_FILLS_COMPLETE poll is mandatory. Never skip it even if the machine believes it has all fills from streaming events.

7.2 Partial fill at entry timeout — residual contracts

Scenario: Entry order partially fills (e.g. 3 of 5 contracts). Timeout fires. Cancel succeeds. Machine moves to ENTRY_RECONCILING. Exchange confirms 3 fills.

Handling: Machine transitions to POSITION_OPEN with netContractsHeld = 3. The accounting layer records the partial position. The strategy is notified via the position_opened_partial_entry event. The hour-harvester strategy must handle a smaller-than-requested position — this is acceptable at launch, since we're working small counts. The exit track exits the actual held count (3), not the originally requested count (5).

Anti-corruption rule: SUBMIT_ORDER for exits always uses ctx.netContractsHeld, not ctx.activeOrder.requestedCount.

7.3 Market expiry during pending exit

Scenario: Exit order is open (EXIT_OPEN). Market resolves before the exit fills. Exchange auto-settles position.

Handling: The engine's market data layer should detect settlement via MARKET_EXPIRY_IMMINENT before this can happen (fire at T-5min). If settlement happens anyway while in EXIT_OPEN or EXIT_PARTIAL:

- Exchange will cancel the open exit order automatically.
- The machine will receive ORDER_CANCEL_OK or a fill event showing 0 remaining.
- RECONCILE_POSITION_COMPLETE will return settled: true and realized_pnl_dollars from the exchange.
- The reconciliation path handles this identically to a normal exit — exchange-authoritative P&L wins.

The engine must poll position settlement status during EXIT_RECONCILING to detect this case.

RECONCILE_POSITION_COMPLETE carries settled: boolean; accounting handles settlement booking via the same ACCOUNTING_RECONCILE_POSITION hook.

7.4 Cancel fails persistently (3 attempts exhausted)

Scenario: `DELETE /orders/{id}` fails 3 times. Machine goes to `TERMINAL_ERROR`.

Handling: This is a human-required situation. The alert fires. The position is likely still open on the exchange. The machine does NOT attempt to place a new exit order from `TERMINAL_ERROR` — that risks doubling the position.

On the next engine restart (after human review), the operator must manually reconcile the position. `ENGINE_RESTORE` into a `TERMINAL_ERROR` state does NOT automatically retry.

Future improvement (post-launch): Add a `MANUAL_OVERRIDE` event that lets an operator signal "position confirmed closed externally" and moves the machine to `TERMINAL_FLAT` with a reconciliation audit trail.

7.5 Duplicate `ORDER_SUBMIT_OK` (exchange replay)

Scenario: Network issues cause the engine to receive two `ORDER_SUBMIT_OK` responses for the same submission. Second one arrives while machine is in `ENTRY_OPEN`.

Handling: The transition function checks `ctx.activeOrder.orderId`. If an order ID is already set and the event carries the same ID, it is a no-op. If it carries a different ID, this is a genuine dual-submit situation — go to `TERMINAL_ERROR` and alert.

Anti-corruption rule: The engine must track submitted request IDs (idempotency keys) at the adapter layer and deduplicate responses before they reach the state machine.

7.6 `STRATEGY_ENTRY_SIGNAL` arrives when not in `IDLE`

Scenario: The strategy emits a signal while the machine is in any non-`IDLE` state.

Handling: The machine silently drops it. The global constraint is one open position max. The engine's outer loop must check `ctx.state === 'IDLE'` before passing a signal to the machine.

Anti-corruption rule: The engine loop MUST gate `STRATEGY_ENTRY_SIGNAL` on `state === 'IDLE'`. Never pass it to the machine in any other state.

7.7 Reprice loop limit exceeded (urgent exit)

Scenario: Urgent exit mode. Market is extremely illiquid. Repriced `urgentMaxReprices` times with no fills.

Handling: On the `urgentRepriceCount >= urgentMaxReprices` guard in `EXIT_REPRICING`, the machine transitions to `TERMINAL_ERROR`, fires `ALERT_HUMAN`, and clears `pendingExitEstimate`. The position may still be live on the exchange.

The position is NOT auto-closed. This is intentional. A reprice loop failure means something is wrong with market conditions or the adapter — automatic further action could make it worse. Human intervention is required.

7.8 `ENGINE_RESTORE` into mid-exit state

Scenario: Engine crashes while in `EXIT_OPEN`. Restarts and loads `execution_context.json`.

Handling: `rehydrateEffects()` inspects the restored state:

- If `EXIT_OPEN` or `EXIT_PARTIAL`: re-register the appropriate timeout (urgent tick or normal timeout), re-schedule a fill poll.
- If `EXIT_CANCEL_PENDING`: immediately retry cancel (attempt count preserved).
- If `EXIT_REPRICING`: immediately re-enter `EXIT_EVALUATING` flow to resubmit.
- If `EXIT_RECONCILING`: immediately re-poll fills and position.

The key property: all rehydration decisions are driven by the persisted state and context. No external state is assumed.

8. Anti-Corruption Rules

1. Never book realized P&L at order placement time. The old repo's `executeExit()` booked P&L at `placeOrder()` time. This is explicitly forbidden. P&L is booked only in `ACCOUNTING_RECONCILE_POSITION` after `EXIT_RECONCILING`.
2. Never use `pendingExitEstimate` for circuit breaker input. It is display-only. `BreakerAccounting` reads only confirmed `LedgerEntry` rows.
3. Never skip `RECONCILE_FILLS_COMPLETE` after a cancel. Late fills are real. Always do a final authoritative fill fetch before settling accounting.
4. Never submit an exit order sized to `activeOrder.requestedCount`. Always use `ctx.netContractsHeld`. These can differ after a partial entry.
5. Never transition out of `TERMINAL_ERROR` automatically. Operator confirmation is required. The machine can only leave this state via `ENGINE_RESTORE` with a validated snapshot that has been corrected by a human.
6. Never pass `STRATEGY_ENTRY_SIGNAL` to the machine when not in `IDLE`. The engine loop is the guard. The machine itself also handles it as a no-op, but the loop must not rely on that.
7. The reducer must be pure. No I/O, no `Date.now()`, no `Math.random()` inside `transition()`. Time and IDs are injected via events or passed as arguments. This ensures the reducer is fully testable.

Implementation note: `Date.now()` appears in the pseudocode above for illustration. In the real implementation, current timestamp should be passed in as a parameter to `transition()` so the function remains pure.
8. Effects are not instructions to the accounting layer to auto-confirm P&L. `ACCOUNTING_RECORD_ENTRY_FILL` creates a fill record and an estimated ledger entry. It does not lock in realized P&L. `ACCOUNTING_BOOK_CYCLE_CLOSED` is the only effect that finalizes a cycle's accounting.
9. Idempotency key on all order submissions. The adapter layer must attach a client-generated idempotency key to every `POST /portfolio/orders` call. Duplicate submits (from rehydration) must not create duplicate orders on the exchange.
10. Fill deduplication is mandatory. Every fill accumulation operation must filter by `fill_id`. The exchange may return the same fill multiple times across polling calls.

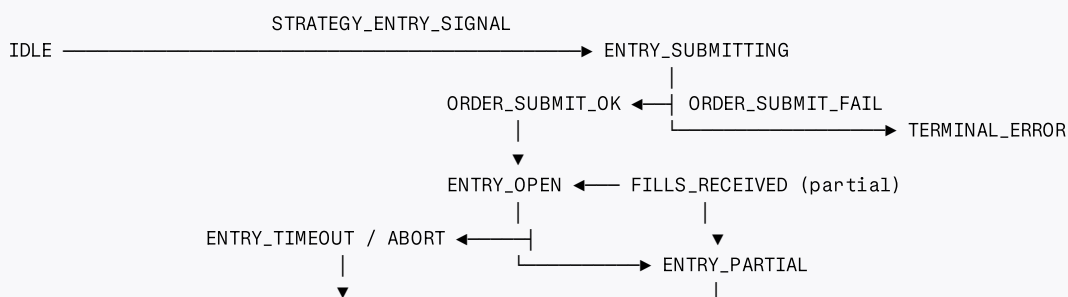
9. Config Surface (Execution Layer Only)

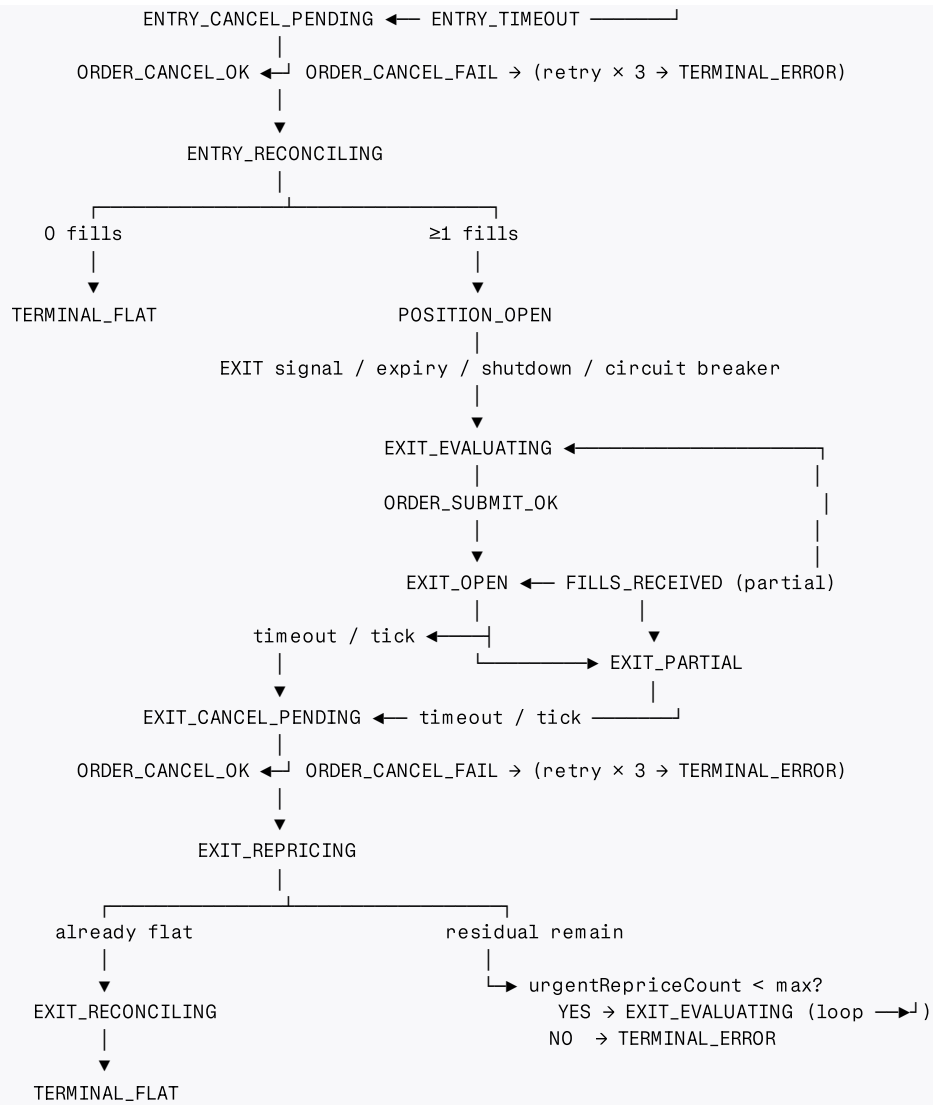
Consistent with the global constraint against settings sprawl, the execution machine requires only these config fields:

```
interface ExecutionConfig {
  entryTimeoutMs: number;           // e.g. 30_000 – cancel entry if unfilled this long
  normalExitTimeoutMs: number;      // e.g. 20_000 – after normal exit order, wait this before repric
  urgentRepricePollMs: number;      // e.g. 3_000 – reprice interval in urgent mode
  urgentMaxReprices: number;        // e.g. 5 – hard cap on reprice attempts before TERMINAL_ERROR
  fillPollIntervalMs: number;      // e.g. 2_000 – standard fill polling interval
  marketExpiryWarningMs: number;   // e.g. 300_000 – fire MARKET_EXPIRY_IMMINENT this many ms before
}
```

Six fields. That's the entire execution configuration surface.

10. State Diagram (ASCII)





End of document. Next pass: implementation build order / module scaffolding.